Luiss
Libera Università Internazionale
degli Studi Sociali Guido Carli

Authentication

Data Privacy and Security

Data Science and Management (DASMA)

LUISS

Authentication Factors

What is Authentication?

The process of reliably verifying the identity of someone (or something).

What is Authentication?

The process of reliably verifying the identity of someone (or something).

Authentication can change depending on who must be authenticated:

What is Authentication?

The process of reliably verifying the identity of someone (or something).

Authentication can change depending on who must be authenticated:

A) **Computers** can:

- store high-quality secrets (long random numbers)
- perform cryptographic operations

What is Authentication?

The process of reliably verifying the identity of someone (or something).

Authentication can change depending on who must be authenticated:

A) **Computers** can:

- store high-quality secrets (long random numbers)
- perform cryptographic operations

B) **Humans** typically remember only passwords, which can be used to:

- Derive a cryptographic key (e.g., hash of password)
- Decrypt a higher quality key locally (e.g., decrypt RSA key)

Authentication Factors

Different approaches to authenticate people:

1. **What you know** (e.g., passwords, PINs)
2. **What you have** (e.g., tokens, smartphones)
3. **Who you are** (e.g., biometrics)
4. **Where you are** (e.g., location-based)

Authentication Factors

Different approaches to authenticate people:

1. **What you know** (e.g., passwords, PINs)
2. **What you have** (e.g., tokens, smartphones)
3. **Who you are** (e.g., biometrics)
4. **Where you are** (e.g., location-based)

These approaches can be combined for **multi-factor authentication (MFA)**.

What You Know

Most common: **passwords** and security questions.

Challenges:

- Often guessable (short, not truly random)
- Can be captured by malware (keyloggers, fake login screens)
- Vulnerable to social engineering
- Users reuse passwords across multiple sites

What You Have

What You Have

Authentication tokens:

- Physical devices performing cryptographic operations
- Generate One-Time Passwords (OTP) valid for limited time

What You Have

Authentication tokens:

- Physical devices performing cryptographic operations
- Generate One-Time Passwords (OTP) valid for limited time

Smartphones:

- Software authentication tokens (general purpose, widespread)
- Receive OTP via SMS (not 100% secure)

What You Have

Authentication tokens:

- Physical devices performing cryptographic operations
- Generate One-Time Passwords (OTP) valid for limited time

Smartphones:

- Software authentication tokens (general purpose, widespread)
- Receive OTP via SMS (not 100% secure)

Open standards for OTP:

- HMAC-based One-Time Password (HOTP)
- Time-based One-Time Password (TOTP)

Who You Are

Biometric authentication:

Who You Are

Biometric authentication:

- **Fingerprint readers:** Widely available on smartphones, but implementation quality varies

Who You Are

Biometric authentication:

- **Fingerprint readers:** Widely available on smartphones, but implementation quality varies
- **Face recognition:** Used in modern smartphones and laptops. Only secure with depth cameras.

Who You Are

Biometric authentication:

- **Fingerprint readers:** Widely available on smartphones, but implementation quality varies
- **Face recognition:** Used in modern smartphones and laptops. Only secure with depth cameras.
- **Retina examination:** Accurate but requires expensive hardware

Who You Are

Biometric authentication:

- **Fingerprint readers:** Widely available on smartphones, but implementation quality varies
- **Face recognition:** Used in modern smartphones and laptops. Only secure with depth cameras.
- **Retina examination:** Accurate but requires expensive hardware
- **Voice recognition:** Active research area, but accuracy affected by health issues and vulnerable to ML-based attacks

Where You Are

Location-based authentication is not considered secure on its own since attackers can often fake location.

However, it can be used as an additional factor:

- Websites warn about logins from unexpected locations
- Banks may refuse transactions from unexpected locations or at unexpected times

Password-Based Authentication

Password Challenges

Common problems:

1. **Robustness:** Human-chosen passwords are weak (short, not random)
2. **Transmission:** Never send passwords in clear (vulnerable to sniffing)
3. **Storage:** Must store passwords securely (encrypted/hashed)

How a website/application should store a password?

Different approaches:

How a website/application should store a password?

Different approaches:

1. *Plain text*: the password is stored in plain text in the database. If the database is leaked, the attacker has access to all passwords! This is not a good idea. **NEVER**.

How a website/application should store a password?

Different approaches:

1. *Plain text*: the password is stored in plain text in the database. If the database is leaked, the attacker has access to all passwords! This is not a good idea. **NEVER**.

2. *Hashed*: the password is hashed before being stored, i.e., $\mathrm{hash}(password)$. The website can still verify if a password is correct by hashing what the user enters and comparing it to the hashed password in the database. However, if the database is leaked, the attacker has access to all hashed passwords, but not the original passwords.

How a website/application should store a password?

Different approaches:

1. *Plain text*: the password is stored in plain text in the database. If the database is leaked, the attacker has access to all passwords! This is not a good idea. **NEVER**.

2. *Hashed*: the password is hashed before being stored, i.e., $\mathrm{hash}(password)$. The website can still verify if a password is correct by hashing what the user enters and comparing it to the hashed password in the database. However, if the database is leaked, the attacker has access to all hashed passwords, but not the original passwords.

3. *Salted* and *Hashed*: the password is hashed with a user-specific random salt before being stored, i.e., $hash(password \,||\, salt)$. The salt is stored with the password in the database. The website can still verify if a password is correct by hashing what the user enters and comparing it to the hashed password in the database. Why? We will understand this in the next slides!

How a website/application should store a password?

Different approaches:

1. *Plain text*: the password is stored in plain text in the database. If the database is leaked, the attacker has access to all passwords! This is not a good idea. **NEVER**.

2. *Hashed*: the password is hashed before being stored, i.e., $\mathrm{hash}(password)$. The website can still verify if a password is correct by hashing what the user enters and comparing it to the hashed password in the database. However, if the database is leaked, the attacker has access to all hashed passwords, but not the original passwords.

3. *Salted* and *Hashed*: the password is hashed with a user-specific random salt before being stored, i.e., $hash(password \,||\, salt)$. The salt is stored with the password in the database. The website can still verify if a password is correct by hashing what the user enters and comparing it to the hashed password in the database. Why? We will understand this in the next slides!

How can we break a hashed password?

Password Attacks

There are three main categories of attacks against passwords:

1. **Brute Force Attack**
2. **Dictionary Attack**
3. **Rainbow Tables**

Each has different characteristics, costs, and countermeasures.

Brute Force Attack: Definition

Exhaustive key search, i.e., we try all possible combinations systematically.

Characteristics:

- Guaranteed to find the password eventually
- Time depends on password space size
- No intelligence or optimization
- Works against any password (weak or strong)

Brute Force Attack: Definition

Exhaustive key search, i.e., we try all possible combinations systematically.

Characteristics:

- Guaranteed to find the password eventually
- Time depends on password space size
- No intelligence or optimization
- Works against any password (weak or strong)

EXAMPLE: For a 4-digit PIN:

- Password space: $10^4 = 10,000$ combinations
- Try: 0000, 0001, 0002, ..., 9999
- Average attempts to find password: $10,000/2 = 5,000$

Brute Force: Password Space

The size of the password space depends on:

- Character set size (alphabet)
- Password length

$$\text{Password Space} = |\text{Alphabet}|^{\text{Length}}$$

Brute Force: Password Space

The size of the password space depends on:

- Character set size (alphabet)
- Password length

$$\text{Password Space} = |\text{Alphabet}|^{\text{Length}}$$

Common character sets:

Character Set	Size	Example
Digits only	10	0-9
Lowercase letters	26	a-z
Upper + lowercase	52	a-z, A-Z
Alphanumeric	62	a-z, A-Z, 0-9
Alphanumeric + symbols	94	a-z, A-Z, 0-9, !@#$...

Brute Force: Time Estimates

Assuming 1 billion (10^9) attempts per second:

Length	Lowercase (26)	Alphanumeric (62)	Full ASCII (94)
4	< 1 second	< 1 second	< 1 second
6	< 1 second	57 seconds	34 minutes
8	5 minutes	2.4 days	60 days
10	3.5 hours	9.5 years	1,500 years
12	100 days	59,000 years	13 million years

Brute Force: Online vs Offline

Brute Force: Online vs Offline

- Online brute force:
  - Attack against live authentication system
  - Limited by network latency and server response time
  - Typical rate: 10-1,000 attempts/second
  - Easily detected and blocked
  - Countermeasures: rate limiting, account lockout, CAPTCHA

Brute Force: Online vs Offline

- Online brute force:
  - Attack against live authentication system
  - Limited by network latency and server response time
  - Typical rate: 10-1,000 attempts/second
  - Easily detected and blocked
  - Countermeasures: rate limiting, account lockout, CAPTCHA

- Offline brute force:
  - Attack against stolen password database
  - Limited only by computational power
  - Modern GPU: billions of hashes/second
  - Hard to detect (attacker has the data)
  - Countermeasures: salting, slow hash functions (PBKDF2, Argon2)

Brute Force: GPU Acceleration

Brute Force: GPU Acceleration

Modern GPUs dramatically accelerate password cracking:

- **Hash rates (single GPU, approximate):**
  - MD5: 200 billion hashes/second
  - SHA-1: 100 billion hashes/second
  - SHA-256: 50 billion hashes/second
  - bcrypt: 100,000 hashes/second
  - Argon2: 10,000 hashes/second

Brute Force: GPU Acceleration

Modern GPUs dramatically accelerate password cracking:

- **Hash rates (single GPU, approximate):**
  - MD5: 200 billion hashes/second
  - SHA-1: 100 billion hashes/second
  - SHA-256: 50 billion hashes/second
  - bcrypt: 100,000 hashes/second
  - Argon2: 10,000 hashes/second

- **Why the difference?**
  - Fast hashes (MD5, SHA) designed for speed
  - Password hashes (bcrypt, Argon2) intentionally slow
  - Memory-hard functions resist GPU parallelization

Dictionary Attack: Definition

Dictionary Attack: Definition

A *smart* brute force where we try only likely passwords from a *dictionary*. Indeed:

Dictionary Attack: Definition

A *smart* brute force where we try only likely passwords from a *dictionary*. Indeed:

A) Humans choose predictable passwords:

- Common words: "password", "admin", "welcome"
- Names: "john", "maria", "alice"
- Dates: "2024", "123456"
- Keyboard patterns: "qwerty", "asdf"

Dictionary Attack: Definition

A *smart* brute force where we try only likely passwords from a *dictionary*. Indeed:

A) Humans choose predictable passwords:

- Common words: "password", "admin", "welcome"
- Names: "john", "maria", "alice"
- Dates: "2024", "123456"
- Keyboard patterns: "qwerty", "asdf"

B) The search becomes sustainable:

- Much smaller search space
- Higher success rate per attempt
- Can crack weak passwords in seconds

Where do dictionaries come from?

Where do dictionaries come from?

1. **Common password lists:**
   - RockYou breach (32 million passwords)
   - LinkedIn breach (117 million passwords)
   - Have I Been Pwned database (billions of passwords)

Where do dictionaries come from?

1. **Common password lists:**
   - RockYou breach (32 million passwords)
   - LinkedIn breach (117 million passwords)
   - Have I Been Pwned database (billions of passwords)

2. **Language dictionaries:**
   - English, Spanish, Chinese words
   - Common phrases and expressions

Where do dictionaries come from?

1. **Common password lists:**
   - RockYou breach (32 million passwords)
   - LinkedIn breach (117 million passwords)
   - Have I Been Pwned database (billions of passwords)

2. **Language dictionaries:**
   - English, Spanish, Chinese words
   - Common phrases and expressions

3. **Targeted dictionaries:**
   - Company-specific terms
   - Industry jargon
   - Personal information (names, dates)

Dictionary Attack: Variations

Attackers apply **rules** to dictionary words:

Dictionary Attack: Variations

Attackers apply **rules** to dictionary words:

- **Common transformations:**
  - Capitalization: password → Password, PASSWORD
  - Leetspeak: password → p@ssw0rd, pa55word
  - Append numbers: password → password1, password123
  - Append symbols: password → password!, password@
  - Combinations: password → P@ssw0rd123!

Dictionary Attack: Variations

Attackers apply **rules** to dictionary words:

- **Common transformations:**
  - Capitalization: password → Password, PASSWORD
  - Leetspeak: password → p@ssw0rd, pa55word
  - Append numbers: password → password1, password123
  - Append symbols: password → password!, password@
  - Combinations: password → P@ssw0rd123!
- **Hybrid attack:** Dictionary + brute force
  - Use dictionary words as base
  - Brute force additional characters
  - Example: "password" + 2 digits = 100 variations

Dictionary Attack: Effectiveness

* https://www.passwordmanager.com/most-common-passwords-latest-statistics/

Dictionary Attack: Effectiveness

Statistics from real breaches:

- Top 10 passwords cover 5% of all users
- Top 100 passwords cover 10% of all users
- Top 10,000 passwords cover 25-30% of all users
- Top 1 million passwords cover 50-60% of all users

* https://www.passwordmanager.com/most-common-passwords-latest-statistics/

Dictionary Attack: Effectiveness

Statistics from real breaches:

- Top 10 passwords cover 5% of all users
- Top 100 passwords cover 10% of all users
- Top 10,000 passwords cover 25-30% of all users
- Top 1 million passwords cover 50-60% of all users

Most common passwords* (2026):

1. 123456
2. 111111
3. admin
4. qwerty
5. password

* https://www.passwordmanager.com/most-common-passwords-latest-statistics/

What about *italian* passwords?

* https://nordpass.com/it/most-common-passwords-list/

What about *italian* passwords?

1. admin
2. password
3. 123456

* https://nordpass.com/it/most-common-passwords-list/

What about *italian* passwords?

1. admin
2. password
3. 123456

8. Napoli1926
9. 123Stella
10. Perlanera

LUISS

* https://nordpass.com/it/most-common-passwords-list/

What about *italian* passwords?

1. admin
2. password
3. 123456

8. Napoli1926
9. 123Stella
10. Perlanera

12. ciaociao

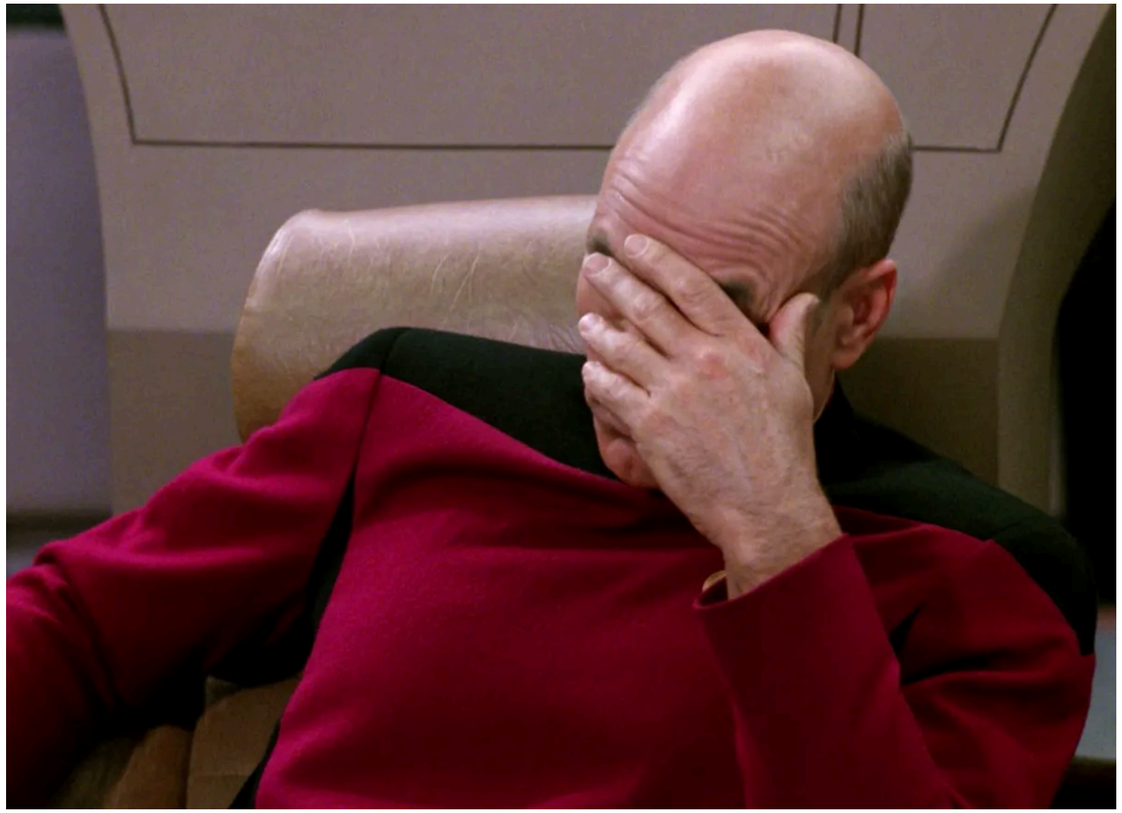
* https://nordpass.com/it/most-common-passwords-list/

What about *italian* passwords?

1. admin
2. password
3. 123456

8. Napoli1926
9. 123Stella
10. Perlanera

12. ciaociao



LUISS

* https://nordpass.com/it/most-common-passwords-list/

Rainbow Tables: Motivation

Rainbow Tables: Motivation

When trying brute force attacks, there is a classic dilemma:

A. **Precompute all hashes:** Fast lookup, but requires massive storage

B. **Compute on-demand:** No storage needed, but very slow

SOLUTION: Store only chain endpoints, not every hash.

Rainbow Tables: How They Work

Chain construction:

1. Start with password, compute hash
2. Apply reduction function R (maps hash → password space)
3. Repeat: hash → reduce → hash → reduce...
4. Store only (start password, end password)

Key insight: One stored pair covers thousands of passwords in the chain.

Rainbow Tables: How They Work

Chain construction:

1. Start with password, compute hash
2. Apply reduction function R (maps hash → password space)
3. Repeat: hash → reduce → hash → reduce...
4. Store only (start password, end password)

Key insight: One stored pair covers thousands of passwords in the chain.

Example chain: `pwd1 → H → hash1 → R → pwd2 → H → hash2 → R → pwd3`

- Store: (pwd1, pwd3)
- This single entry covers pwd1, pwd2, pwd3 and their hashes.

Rainbow Tables: Lookup

Given hash to crack:

1. Apply reduction function and check if result matches any chain end
2. If match found, regenerate chain from start to find the password
3. If no match, apply hash then reduction and check again
4. Repeat until found or exhausted

Trade-off: Small storage, moderate lookup time.

Rainbow Tables: Storage savings

Method	Storage	Lookup
Full table	100%	Instant
Rainbow table	0.1%	Moderate
No precomputation	0%	Very slow

Rainbow Tables: The Defense

Rainbow Tables: The Defense

Salting before hashing completely defeats rainbow tables!

Indeed, when using a salt:

$$h = H(\text{password} \,||\, \text{salt})$$

Each unique and user-specific salt requires a different rainbow table, making the precomputation of rainbow tables infeasible.

Why salting works

- 64-bit salt = 2^{64} possible salts
- Would need 2^{64} different rainbow tables
- Storage becomes impractical: 64 GB $\times$ 2^{64} = 10^{21} GB

CONCLUSION: Always use salting to protect against rainbow tables.

Attack Comparison Summary

Attack Comparison Summary

Attack	Speed	Storage	Success Rate	Defeated By
Brute Force	Slow	None	100%	Long passwords
Dictionary	Fast	Small	50-60%	Random passwords
Rainbow Tables	Very Fast	Large	High	Salting

Attack Comparison Summary

Attack	Speed	Storage	Success Rate	Defeated By
Brute Force	Slow	None	100%	Long passwords
Dictionary	Fast	Small	50-60%	Random passwords
Rainbow Tables	Very Fast	Large	High	Salting

Defense strategy:

1. Use salting (defeats rainbow tables)
2. Use slow hash functions (slows brute force)
3. Enforce strong password policies (defeats dictionary)
4. Implement rate limiting (slows online attacks)
5. Use multi-factor authentication (adds extra layer)